HABIB UNIVERSITY

School of Science & Engineering
Department of Electrical Engineering

Project Report

Course	EE/CE 321L / 371L – Computer Architecture Lab
Semester	Spring 2026
Instructor	Professor Waseem Hassan Research Assistant - Muddassir Ali Siddiqui
Lab Section	T1

Group Members

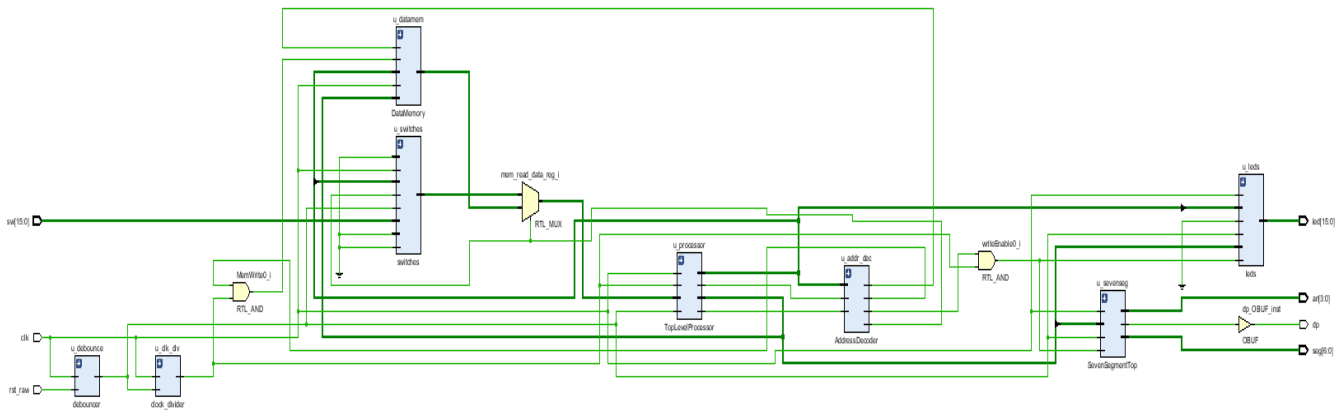
Amna Ali	aa09863
Human Khan	hk09629

Git Repository URL: Link

Task A: Base Assembly Program

This section covers the RISC-V assembly program developed in Lab 10, loaded into the instruction memory of the integrated single-cycle RISC-V processor from Lab 11 and executed on hardware.

A.1 RTL



A.2 Assembly Code

Code | Blame | 24 lines (22 loc) · 569 Bytes

```
#   x6  - switch value read / temp
#   x7  - countdown counter
#   x8  - switch base address  (0x300 = 768)
#   x9  - LED base address     (0x200 = 512)


_start:
    addi x8, x0, 768
    addi x9, x0, 512
WAIT:
    sw   x0, 0(x9)
WAIT_ZERO:
    lw   x6, 0(x8)
    bne  x6, x0, WAIT_ZERO    # ← BNE here
WAIT_POLL:
    lw   x6, 0(x8)
    beq  x6, x0, WAIT_POLL    # ← BEQ here (not BNE)
    sw   x6, 0(x9)
    addi x7, x6, 0
COUNT_LOOP:
    beq  x7, x0, WAIT         # ← BEQ here
    sw   x7, 0(x9)
    addi x7, x7, -1
    beq  x0, x0, COUNT_LOOP   # ← BEQ here
```

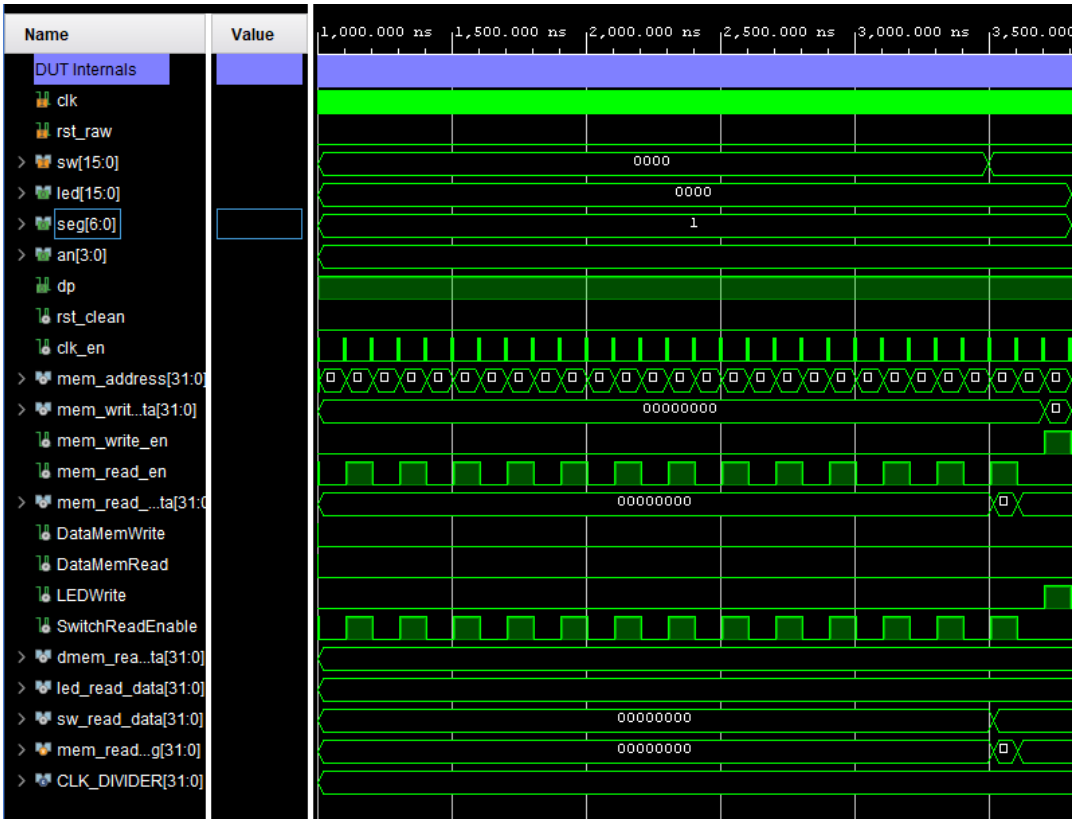
A.3 Instruction Memory Initialization (.mem File)

```
30000413
20000493
0004a023
00042303
fe031ee3
00042303
fe030ee3
0064a023
00030393
fe0382e3
0074a023
fff38393
fe000ae3
```

A.4 Testbench (tb)

```
`timescale 1ns / 1ps

module tb_ProcessorFPGA;
    reg         clk;
    reg         rst_raw;
    reg  [15:0] sw;
    wire [15:0] led;
    wire [6:0]  seg;
    wire [3:0]  an;
    wire        dp;
    ProcessorFPGA uut (
        .clk     (clk),
        .rst_raw (rst_raw),
        .sw      (sw),
        .led     (led),
        .seg     (seg),
        .an      (an),
        .dp      (dp)
    );
    defparam uut.u_clk_div.MAX_COUNT   = 10;   // slow_clk every 10 cycles
    defparam uut.u_debounce.STABLE_MAX = 10;   // debouncer fires after 10 cycles
    initial clk = 0;
    always #5 clk = ~clk;
    initial begin
        rst_raw = 1;
        sw      = 16'd0;
        repeat(50) @(posedge clk);   // 50 cycles >> STABLE_MAX=10, debouncer fires
        rst_raw = 0;
        repeat(300) @(posedge clk);  // polling phase, sw=0
        sw = 16'd5;
        repeat(2000) @(posedge clk); // countdown from 5
        sw = 16'd0;
        repeat(300) @(posedge clk);
        sw = 16'd3;
        repeat(1500) @(posedge clk); // countdown from 3
        sw = 16'd0;
        repeat(300) @(posedge clk);
        $finish;
    end
endmodule
```

A.5 Simulation Waveforms


Name
Value
1,000.000 ns
1,500.000 ns
2,000.000 ns
2,500.000 ns
3,000.000 ns
DUT Internals
clk
rst_raw
sw[15:0]
led[15:0]
seg[6:0]
an[3:0]
dp
rst_clean
clk_en
mem_address[31:0]
mem_writ...ta[31:0]
mem_write_en
mem_read_en
mem_read_...ta[31:0]
DataMemWrite
DataMemRead
LEDWrite
SwitchReadEnable
dmem_rea...ta[31:0]
led_read_data[31:0]
sw_read_data[31:0]
mem_read...g[31:0]
CLK_DIVIDER[31:0]
0000
0000
1
00000000
00000000
00000000
00000000

DUT Internals
clk 1
rst_raw 0
sw[15:0] 0005
led[15:0] 0005
seg[6:0] 36
an[3:0] e
dp 1
rst_clean 0
clk_en 0
mem_address[31:0] 00000004
mem_writ...ta[31:0] 00000000
mem_write_en 0
mem_read_en 0
mem_read_...ta[31:0] 00000000
DataMemWrite 0
DataMemRead 0
LEDWrite 0
SwitchReadEnable 0
dmem_rea...ta[31:0] 00000000
led_read_data[31:0] 00000000
sw_read_data[31:0] 00000005
mem_read...g[31:0] 00000000
CLK_DIVIDER[31:0] 017d7840
0005 0004 0003 0002 0001
36 76 6 18 79
00000000
00000000

Task B: Instruction Extension

Three new RISC-V instructions of different types are added to the processor. Each instruction is described below with the required datapath/control modifications, assembly test, and verification.

B.1 Additional Instruction 1

Instruction: BNE (B-type)

Opcode / funct3: 1100011 / 001

B.2 Additional Instruction 2

Instruction: JAL (J type)

Opcode / funct3: 1101111

B.3 Additional Instruction 3

Instruction: JALR (I type)

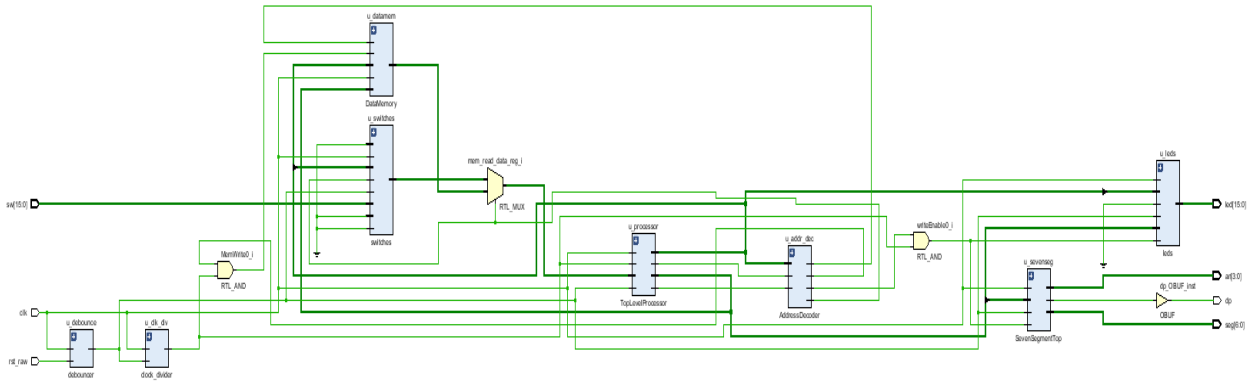
Opcode / funct3 : 1100111/000

B.4 RTL – Control & Datapath Modifications

MainControl.v — two new output signals were added: Jump (HIGH for JAL) and JumpReg (HIGH for JALR). JAL sets RegWrite=1 and Jump=1 so the return address gets written to rd. JALR sets RegWrite=1, ALUSrc=1, and JumpReg=1 so the ALU computes rs1+imm as the jump target. BNE required no changes — it already worked through the existing Branch signal combined with funct3=001 in the branch condition logic.

TopLevelProcessor.v — the PC-next mux was extended from 2 to 3 inputs: normal PC+4, branch/JAL target from the branch adder (PC+imm), and JALR target from the ALU result (rs1+imm). The writeback mux was similarly extended — when Jump or JumpReg is HIGH, PC+4 is written to rd as the return address instead of the ALU result or memory data.

immGen.v — J-type immediate decoding was added for JAL, reassembling the scrambled instruction bits [31,19:12,20,30:21] into a correct signed 21-bit offset.



B.5 Assembly Test Code

```
# Part B Test Program - Tests BNE, JAL, JALR

# BNE TEST
addi x1, x0, 5        # x1 = 5
addi x2, x0, 3        # x2 = 3
bne  x1, x2, +8       # BNE: 5!=3 TAKEN, skip next instruction
addi x3, x0, 0xBAD    # SKIPPED (proves BNE jumped)
addi x4, x0, 0x11     # x4=0x11, BNE landed here

# JAL TEST
jal  x5, +12          # JAL: x5=PC+4=0x18, jump to 0x20
addi x6, x0, 0xBAD    # SKIPPED
addi x6, x0, 0xBAD    # SKIPPED
addi x7, x0, 0x22     # x7=0x22, JAL landed here

# JALR TEST
addi x8, x0, 0x38     # x8 = 0x38 (target address)
jalr x9, 0(x8)        # JALR: x9=PC+4=0x2C, jump to x8=0x38
addi x10, x0, 0xBAD   # SKIPPED
add x0,x0,x0          # gap
add x0,x0,x0          # gap
addi x10, x0, 0x33    # x10=0x33, JALR landed here

# HALT
beq  x0, x0, 0        # infinite loop
```

B.6 Memory / .mem File

```
00500093
```

```
00300113
00209463
bad00193
01100213
00c002ef
bad00313
bad00313
02200393
03800413
000404e7
bad00513
00000013
00000013
03300513
00000063
```

B.7 Testbench

```
`timescale 1ns / 1ps



module tb_PartB;


    reg         clk;
    reg         rst;
    wire [31:0] mem_address;
    wire [31:0] mem_write_data;
    wire        mem_write_en;
    wire        mem_read_en;
    reg  [31:0] mem_read_data;

    // Instantiate TopLevelProcessor directly
    // clk_en tied HIGH - every cycle executes in simulation
    // No slow clock needed - we step cycle by cycle
    TopLevelProcessor uut (
        .clk            (clk),
        .clk_en         (1'b1),
        .rst            (rst),
        .mem_address    (mem_address),
        .mem_write_data (mem_write_data),
        .mem_write_en   (mem_write_en),
        .mem_read_en    (mem_read_en),
        .mem_read_data  (mem_read_data)
    );
```

```
    // Simple data memory model inside testbench
    // 256 words, word addressed via address[9:2]
    reg [31:0] dmem [0:255];
    integer mi;

    initial begin
        for (mi = 0; mi < 256; mi = mi + 1) begin
            dmem[mi] = 32'd0;
        end
    end

    always @(posedge clk) begin
        if (mem_write_en) begin
            dmem[mem_address[9:2]] <= mem_write_data;
        end
    end

    always @(*) begin
        if (mem_read_en) begin
            mem_read_data = dmem[mem_address[9:2]];
        end
        else begin
            mem_read_data = 32'd0;
        end
    end

    // Clock - 10ns period
    initial begin
        clk = 1'b0;
    end

    always begin
        #5 clk = ~clk;
    end

    // Test sequence
    initial begin

        // Reset for 3 cycles
        rst = 1'b1;
        repeat(3) @(posedge clk);
        rst = 1'b0;

        // Run 40 cycles - enough for all 16 instructions
        // including the HALT loop at 0x3C
        repeat(40) @(posedge clk);

        $finish;
    end

endmodule
```

B.8 Simulation Waveforms

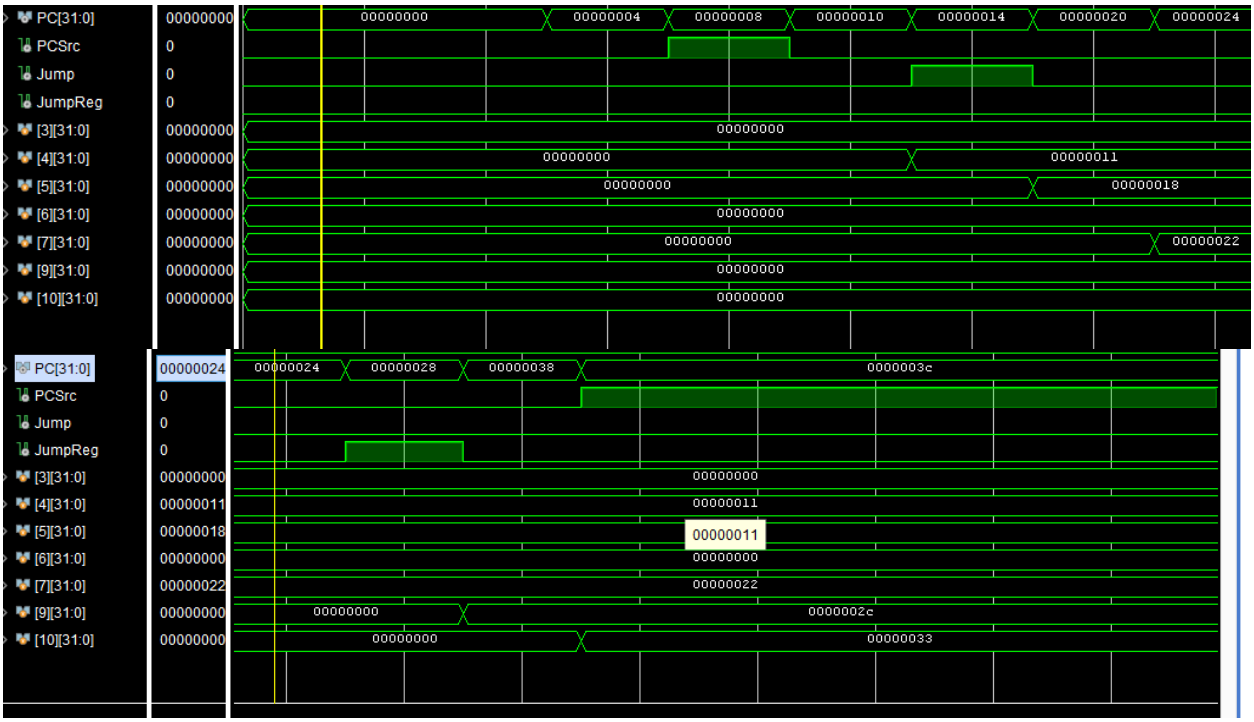


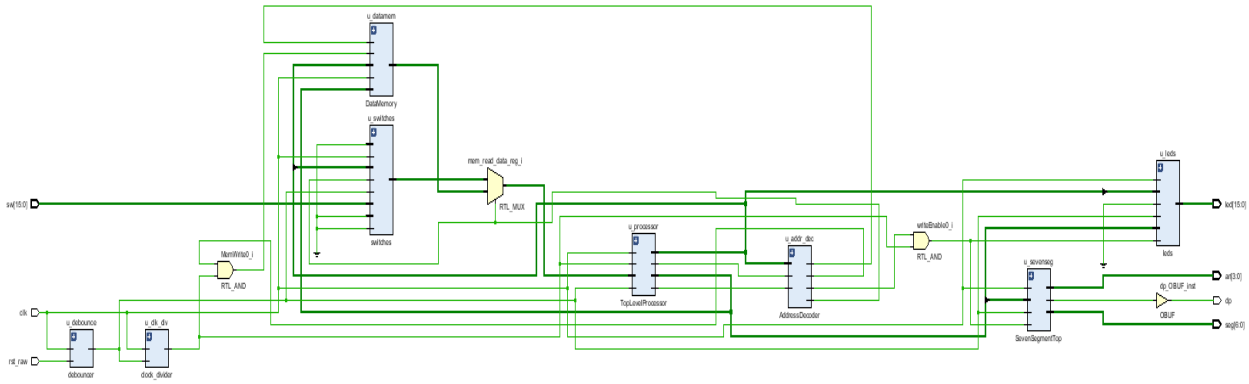
Fig 1.1: PC jumps non-sequentially from 0x08→0x10 (BNE taken), 0x14→0x20 (JAL fired, Jump=HIGH) and 0x28→0x38 (JALR fired, JumpReg=HIGH). Skipped registers x3 and x6 remain 0x00000000 while target registers x4=0x11, x5=0x18, x7=0x22, x9=0x2C and x10=0x33 confirm all three instructions execute and redirect the PC correctly.

Task C: Program Demonstration on Hardware

A meaningful RISC-V assembly program is demonstrated on the FPGA-based processor. The program makes use of stack operations, procedure calls, and register preservation.

Program Name / Type: *Fibonacci Series*

C.1 RTL



C.2 Assembly Code

```
addi x2,  x0,  508   # sp = 0x1FC (stack pointer)
addi x8,  x0,  768   # x8 = 0x300 (N address, reads switches)
addi x9,  x0,  512   # x9 = 0x200 (LED address)
sw   x0,  0(x9)      # show 0 on LEDs

lw   x10, 0(x8)      # read N from switches
bne  x10, x0, -4     # wait until N != 0

lw   x10, 0(x8)      # read N again
beq  x10, x0, -4     # wait until switches released

sw   x10, 0(x9)      # show N on LEDs
JAL  x1, +8          # call fib_procedure
BEQ  x0, x0, -28     # loop back to start

# fib_procedure:
addi x2,  x2, -16    # allocate stack
sw   x1, 12(x2)      # save ra
sw   x9,  8(x2)      # save x9
sw   x8,  4(x2)      # save x8
sw   x7,  0(x2)      # save x7

addi x5,  x0,  0     # a = 0 (fib[0])
addi x6,  x0,  1     # b = 1 (fib[1])
addi x7,  x10, 0     # counter = N

beq  x7,  x0, done   # if N==0 skip
sw   x5,  0(x9)      # show a on LEDs
add  x11, x5,  x6    # x11 = a+b
addi x5,  x6,  0     # a = b
addi x6,  x11, 0     # b = a+b
addi x7,  x7,  -1    # counter--
bne  x7,  x0,  -24   # if not done loop back
```

```
# epilogue:
lw   x7,  0(x2)      # restore x7
lw   x8,  4(x2)      # restore x8
lw   x9,  8(x2)      # restore x9
lw   x1,  12(x2)     # restore ra
addi x2,  x2, +16    # deallocate stack
JALR x0,  x1, 0      # return
```

C.3 Instruction Memory Initialization (.mem File)

```
1fc00113
30000413
20000493
0004a023
00042503
fe051ee3
00042503
fe050ee3
00a4a023
008000ef
fe0002e3
ff010113
00112623
00912423
00812223
00712023
00000293
00100313
00050393
00038e63
0054a023
006285b3
00030293
00058313
fff38393
fe0394e3
00012383
00412403
00812483
00c12083
01010113
00008067
```

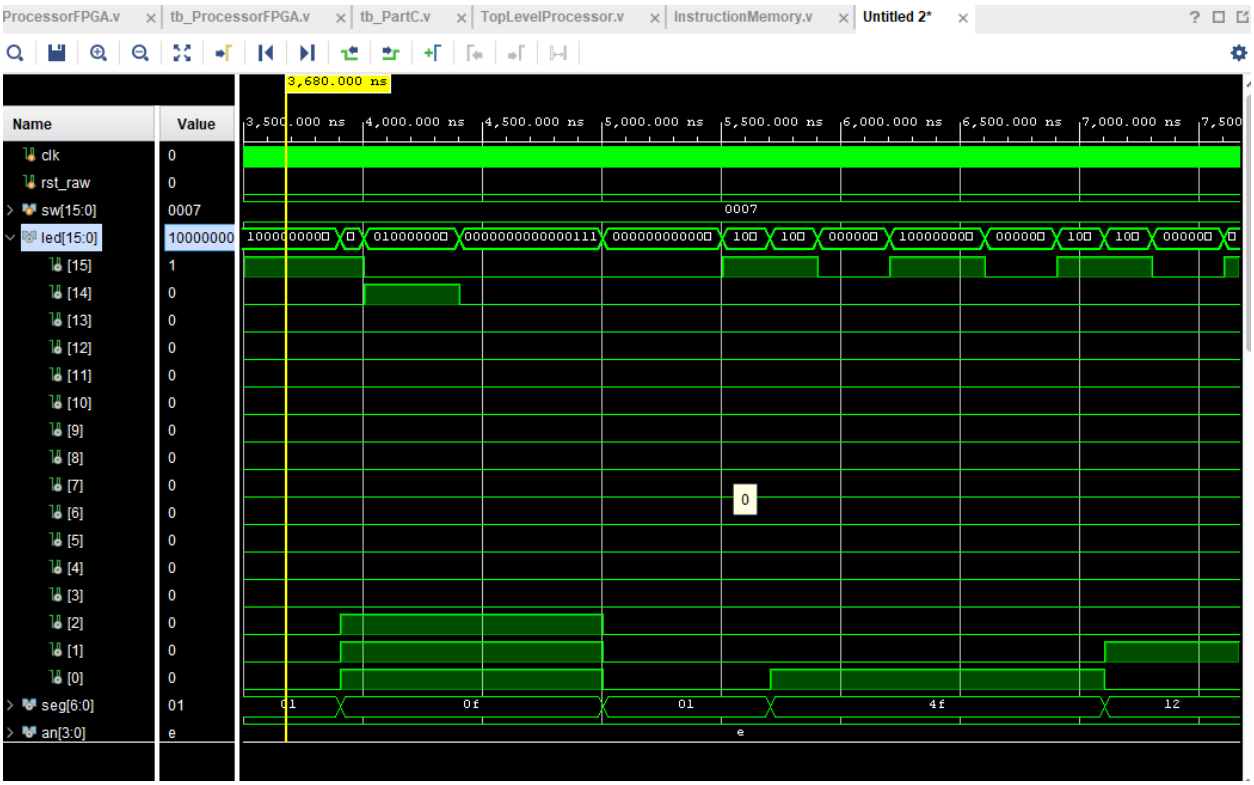
C.4 Testbench

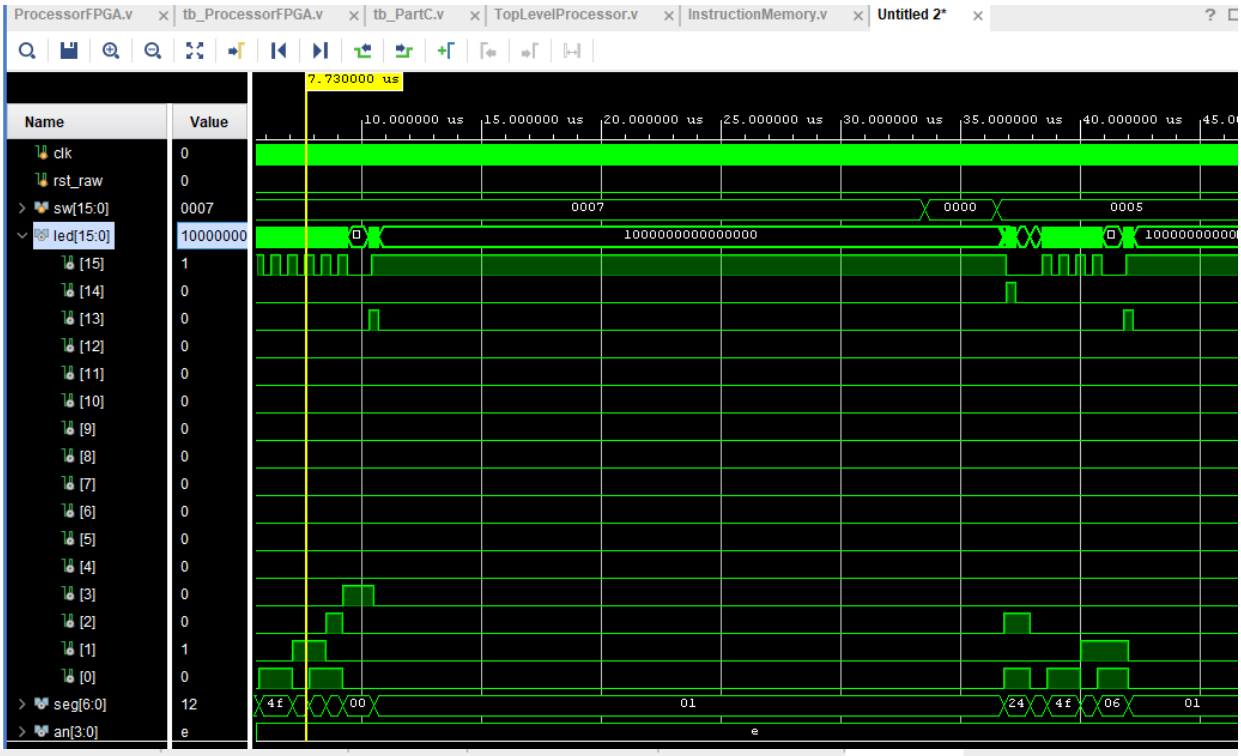
```
`timescale 1ns / 1ps
```

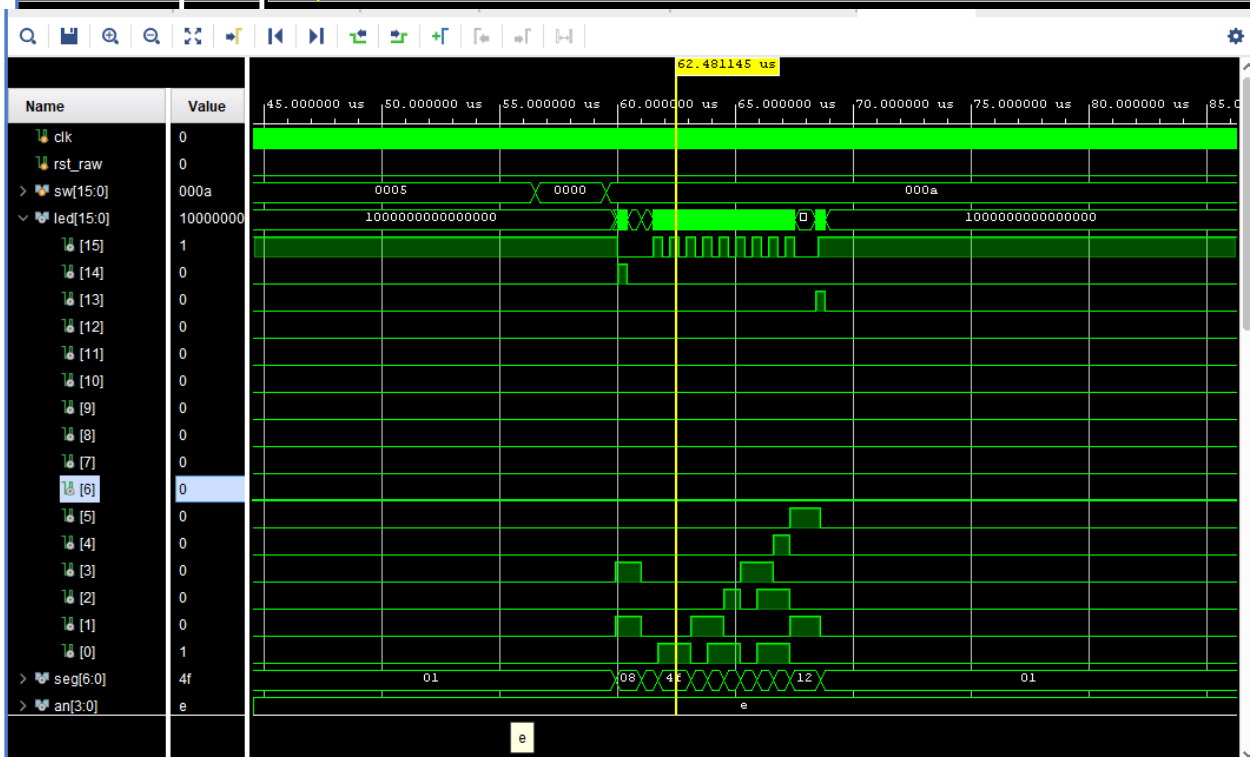
```
module tb_PartC;

    reg        clk;
    reg        rst_raw;
    reg  [15:0] sw;
    wire [15:0] led;
    wire [6:0]  seg;
    wire [3:0]  an;
    wire       dp;

    ProcessorFPGA uut (
        .clk     (clk),
        .rst_raw (rst_raw),
        .sw      (sw),
        .led     (led),
        .seg     (seg),
        .an      (an),
        .dp      (dp)
    );

    // Speed up for simulation - same trick as Part A testbench
    defparam uut.u_clkdiv.MAX_COUNT    = 10;   // clk_en every 10 cycles
    defparam uut.u_debounce.STABLE_MAX = 10;   // debouncer fires after 10 cycles

    initial clk = 0;
    always #5 clk = ~clk;

    initial begin
        rst_raw = 1;
        sw      = 16'd0;

        // Hold reset well above STABLE_MAX=10 so debouncer fires
        repeat(50) @(posedge clk);
        rst_raw = 0;

        // Wait for debouncer to release rst and processor to start
        repeat(300) @(posedge clk);

        // -------------------------------------------------------
        // TEST 1 - N=7
        // Expected LED sequence: 0,1,1,2,3,5,8,13
        // Watch u_leds/leds change in waveform
        // -------------------------------------------------------
        sw = 16'd7;
        repeat(3000) @(posedge clk);

        // Pull switches low - processor returns to WAIT_ZERO
        sw = 16'd0;
        repeat(300) @(posedge clk);

        // -------------------------------------------------------
```

```
        // TEST 2 - N=5
        // Expected LED sequence: 0,1,1,2,3,5
        // -------------------------------------------------------
        sw = 16'd5;
        repeat(2000) @(posedge clk);

        sw = 16'd0;
        repeat(300) @(posedge clk);

        // -------------------------------------------------------
        // TEST 3 - N=10
        // Expected LED sequence: 0,1,1,2,3,5,8,13,21,34,55
        // -------------------------------------------------------
        sw = 16'd10;
        repeat(5000) @(posedge clk);

        sw = 16'd0;
        repeat(300) @(posedge clk);

        $finish;
    end

    // Timeout guard
    initial begin
        #200000;
        $finish;
    end

endmodule
```

C.5 Simulation Waveforms


ProcessorFPGA.v
tb_ProcessorFPGA.v
tb_PartC.v
TopLevelProcessor.v
InstructionMemory.v
Untitled 2*
3,680.000 ns
Name
Value
clk
rst_raw
sw[15:0]
led[15:0]
seg[6:0]
an[3:0]
0007
10000000
01
0f
4f
12
e


ProcessorFPGA.v
tb_ProcessorFPGA.v
tb_PartC.v
TopLevelProcessor.v
InstructionMemory.v
Untitled 2*
7.730000 us
Name
Value
clk 0
rst_raw 0
sw[15:0] 0007
led[15:0] 10000000
[15] 1
[14] 0
[13] 0
[12] 0
[11] 0
[10] 0
[9] 0
[8] 0
[7] 0
[6] 0
[5] 0
[4] 0
[3] 0
[2] 0
[1] 1
[0] 0
seg[6:0] 12
an[3:0] e
10.000000 us
15.000000 us
20.000000 us
25.000000 us
30.000000 us
35.000000 us
40.000000 us
0007
0000
0005
1000000000000000
4f
00
01
24
4f
06
01
e


62.481145 us
Name
Value
clk 0
rst_raw 0
sw[15:0] 000a
led[15:0] 10000000
[15] 1
[14] 0
[13] 0
[12] 0
[11] 0
[10] 0
[9] 0
[8] 0
[7] 0
[6] 0
[5] 0
[4] 0
[3] 0
[2] 0
[1] 0
[0] 1
seg[6:0] 4f
an[3:0] e
45.000000 us
50.000000 us
55.000000 us
60.000000 us
65.000000 us
70.000000 us
75.000000 us
80.000000 us
0005
0000
000a
1000000000000000
01
08
4f
12
01
e

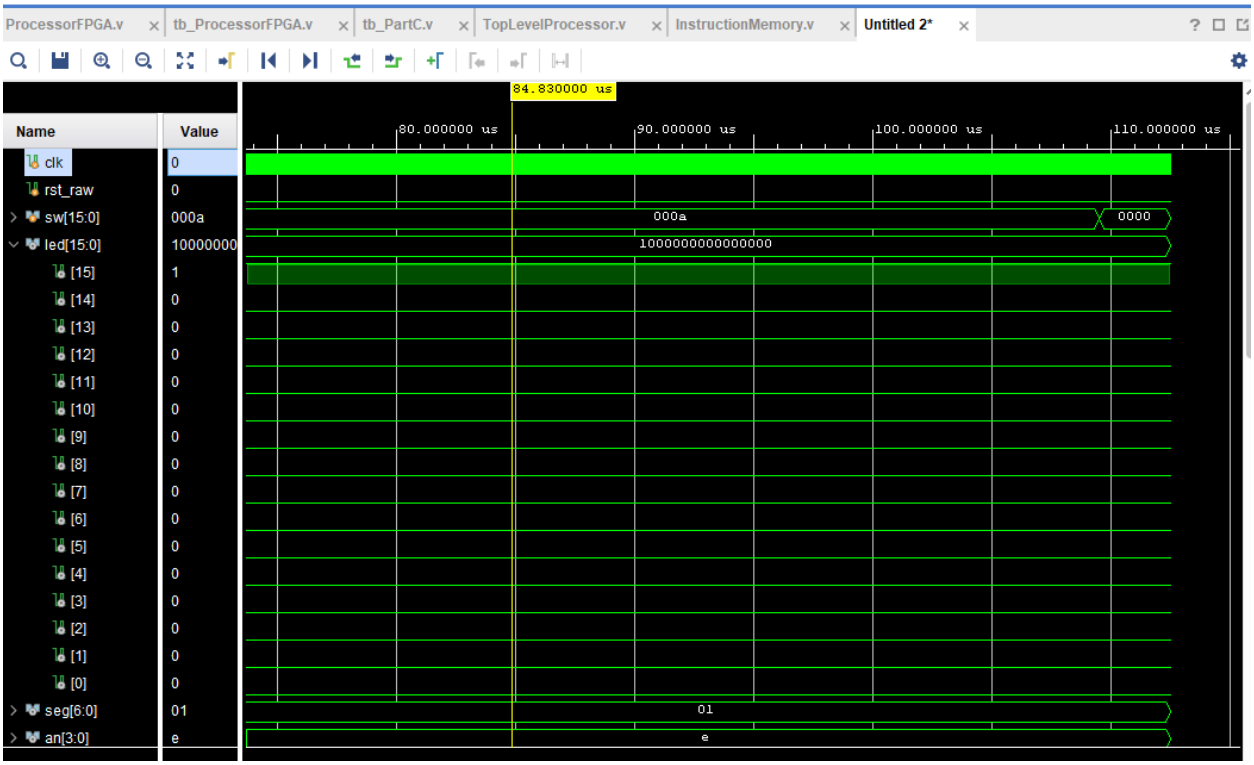


Fig 1.2: led[15] bne , led[14] jal and led[13] jalr. First switch input is displayed followed by the Fibonacci series up to N sw[15:0]